INDIVIDUAL TASK

Design and Implementation of a Perceptron Model for Binary Classification

ABSTRACT

A perceptron is one of the fundamental models of Artificial Neural Networks used for solving binary classification problems. This individual task focuses on designing and implementing a single-layer perceptron model to classify input data into two distinct classes. The perceptron is constructed using input features, associated weights, a bias term, and a step activation function. The Perceptron Learning Law is applied to iteratively update the weights and bias based on the classification error.

The task is carried out manually using paper or a spreadsheet to clearly understand the learning mechanism of the perceptron. A simple linearly separable dataset is considered, and the training process is performed step by step until the perceptron correctly classifies all input patterns. The results demonstrate that the perceptron successfully learns the decision boundary for binary classification problems. This experiment provides a strong foundation for understanding neural network learning and forms the basis for more advanced machine learning models.

1. Introduction

Artificial Neural Networks (ANNs) are inspired by the working of the human brain and are widely used in machine learning and pattern recognition. One of the earliest and simplest neural network models is the Perceptron, proposed to perform binary classification tasks.

A perceptron acts as a single artificial neuron that learns to separate input data into two distinct classes by adjusting its weights based on errors. This individual task focuses on manually building a perceptron model and applying the Perceptron Learning Law to understand how learning occurs in neural networks.

2. Aim of the Task

The aim of this task is to:

- Understand the basic structure of a perceptron
- Design a perceptron model for a binary classification problem
- Apply the perceptron learning rule manually
- Analyze how weights and bias change during training
- Verify the correctness of classification after training

3. Scope of the Task

This task is limited to:

- Single-layer perceptron
- Binary input and binary output
- Linearly separable data
- Manual calculation using paper or spreadsheet

It does not include multi-layer networks or non-linear activation functions.

4. Theoretical Background

4.1 Artificial Neuron

An artificial neuron receives inputs, multiplies them by weights, adds a bias, and passes the result through an activation function to produce an output.

4.2 Perceptron

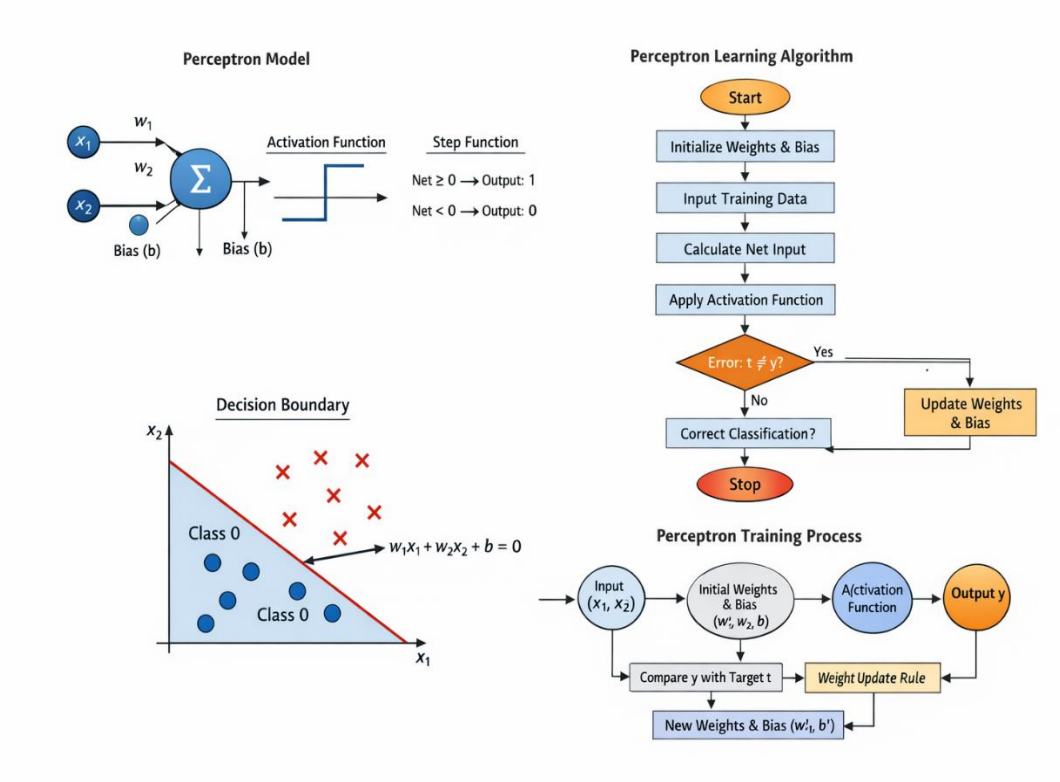
A perceptron is a single-layer neural network capable of solving only linearly separable problems. It cannot solve non-linear problems such as XOR.

5. Architecture of the Perceptron

The perceptron architecture consists of:

- Input layer (features)
- Weight vector
- Bias unit
- Summation unit

- Activation function
- Output layer



6. Mathematical Model

6.1 Net Input Calculation

$$\text{Net Input} = \sum_{i=1}^{n} w_i\, x_i + b$$

Where:

- x_i= input values
- w_i= corresponding weights
- b= bias

6.2 Activation Function

A step function is used:

$$f(net) = \begin{cases} 1, & \text{if } net \geq 0 \\ 0, & \text{if } net < 0 \end{cases}$$

7. Problem Statement

The task is to design a perceptron that performs binary classification for a given dataset. The perceptron must learn from the training data and adjust its weights until it classifies all inputs correctly.

8. Dataset Description

The dataset selected represents a simple logical AND operation:

Input x_1	Input x_2	Target Output
0	0	0
0	1	0
1	0	0
1	1	1

This dataset is chosen because it is linearly separable, making it suitable for a perceptron model.

9. Initial Assumptions

- Initial weights are set to zero
- Bias is initialized to zero
- Learning rate is set to 1
- Training is performed iteratively over the dataset

10. Perceptron Learning Law

The perceptron learning algorithm updates weights based on classification error.

Weight Update Formula

$$w_i^{new} = w_i^{old} + \eta(t - y)x_i$$

Bias Update Formula

$$b^{new} = b^{old} + \eta(t - y)$$

Where:

- t= target value
- y= predicted output
- η= learning rate

11. Training Algorithm

1. Initialize weights and bias
2. Select an input pattern
3. Compute net input
4. Apply activation function
5. Compare predicted output with target
6. Update weights and bias if error exists
7. Repeat for all inputs
8. Continue training until no errors occur

12. Manual Training Process

During training:

- When the perceptron output matches the target, no update is required
- When the output differs from the target, weights and bias are updated
- Multiple epochs may be required for convergence

The perceptron gradually learns the correct decision boundary.

13. Decision Boundary

The learned weights define a straight line that separates the two classes. This line is known as the decision boundary. All inputs on one side of the line belong to Class 0, and inputs on the other side belong to Class 1.

14. Final Trained Model

After training, the perceptron produces correct outputs for all input combinations, confirming successful learning.

15. Results

- The perceptron correctly classifies all training samples
- The model converges within a finite number of iterations
- Learning is successful due to linear separability of data

16. Advantages of Perceptron

- Simple and easy to implement
- Fast learning process
- Suitable for binary classification
- Forms the foundation of neural networks

17. Limitations of Perceptron

- Cannot solve non-linearly separable problems
- Limited to binary output
- Not suitable for complex datasets

18. Applications

- Logical gate implementation
- Image pattern recognition

- Spam detection
- Decision-making systems

19. Conclusion

This individual task successfully demonstrated the construction and training of a perceptron model for binary classification. By applying the perceptron learning law, the model learned to adjust its weights and bias to minimize classification error. This task provides a strong foundation for understanding advanced neural network architectures.

20. Future Enhancements

- Extension to multi-layer perceptrons
- Use of different activation functions
- Application to real-world datasets